

AP 2026 - Assignment 2

Lukas Vammen - TQR452
Jimmy Huynh - PLG852
Karan Inder Singh - GTC626

September 20, 2026

Contents

Introduction: 3

Questions and Answers: 4

 Question 1 4

 Question 2 4

 Question 3 4

 Question 4 5

Introduction:

The handin consists of a solution for all tasks provided.

The implementation for the requisite functions has been completed and the code builds without warnings.

AI was used in the structuring and creation of the tests.

We are fairly certain that the solution provided is complete and have good reason to believe so: The well structured and rigorous test suite covers edge cases and provides an explicit proof of the same.

Questions and Answers:

1. What is the asymptotic complexity of your implementation of `Print`? Can this be improved?

Answer:

- **Current Complexity:** $\mathcal{O}(n)$ per `evalPrint` operation, where n is the number of strings currently accumulated in `output`. This leads to an overall complexity of $\mathcal{O}(k^2)$ for k print statements. List concatenation (`++`) must traverse the entire left-hand list `output` to append `[str]` at the end.
- **Improvement:** Prepend new elements to the front of the list using `cons (:) (str : output)` in $\mathcal{O}(1)$ time, and perform a single `reverse` on `output` at the end of execution. This brings the overall complexity down to $\mathcal{O}(k)$ (or $\mathcal{O}(1)$ amortized per print operation).

2. In your implementation, if a program keeps updating the same key with `KvPut`, does the memory usage of the program remain constant or does it increase?

Answer:

The memory usage of the store remains **constant** when repeatedly updating the same key.

In `evalKvPut`, the function purges any existing entry for the key before inserting the new value:

```
filter (\(k, _) -> k /= v1) store ++ [(v1, v2)]
```

Because matching keys are filtered out rather than appended alongside old entries, the store list does not grow over time when the same key is updated.

3. In `TryCatch e1 e2`, if `e1` fails after performing some effects, are those effects visible in `e2`, and which of your tests demonstrate this? If you wanted different behaviour (either making the effects visible or invisible), what would you need to change in your implementation? Show exactly what the new code would look like.

Answer:

No, the effects performed by `e1` before it fails are **not** visible in `e2`.

In the implementation of `catch`, the state returned by `m1` is discarded upon failure:

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->
  case m1 env state of
    (_, Left _) -> m2 env state
    (state', Right x) -> (state', Right x)
```

When `m1 env state` evaluates to `(state1, Left err)`, the updated state `state1` is matched as `_` and ignored. Instead, `m2` is executed with the original `state` that was passed into `catch`.

2. Test Case Demonstration

A test demonstrates this by performing a side effect (such as writing to the key-value store or printing) in `e1` right before raising an error, and then attempting to access that state in `e2`:

-- Example test expression:

```
TryCatch
  (Let "x" (KvPut (CstInt 1) (CstInt 100)) (Div (CstInt 1) (CstInt 0)))
  (KvGet (CstInt 1))
```

Behavior:

- `e1` stores `(1, 100)` in the key-value store, but subsequently fails with a division-by-zero error.
- Because state modifications from `e1` are discarded, `e2` runs with an empty key-value store.

- `KvGet (CstInt 1)` fails with `"Invalid key: ValInt 1"`. If effects were visible, it would succeed and return `ValInt 100`.

3. Implementation Change for Visible Effects

Currently, the implementation enforces **invisible** effects (state rollback on failure). To make side effects from `e1` **visible** in `e2` (and preserved in the final state), pass the modified state `state1` to `m2` (as well as returning `state1` on success):

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->
  case m1 env state of
    (state1, Left _) -> m2 env state1
    (state', Right x) -> (state', Right x)
```

4. Would it be possible to define type class instances

```
instance Monad m => Semigroup (m ()) where
  ...
```

```
instance Monad m => Monoid (m ()) where
  ...
```

and what might the definitions for `mempty` and `(<>)` look like? If so, argue based on the monad laws that your instances satisfy the monoid laws.

Definitions

Defining these instances directly requires GHC language extensions (`FlexibleInstances` and `OverlappingInstances`) to resolve type-checker conflicts:

```
{-# LANGUAGE FlexibleInstances, OverlappingInstances #-}
```

```
instance Monad m => Semigroup (m ()) where
  a <> b = a >>= \_ -> b
```

```
instance Monad m => Monoid (m ()) where
  mempty = return ()
```

Proof of Monoid Laws

Using the standard Monad laws:

1. **Left Identity:** $\text{return } x \gg= f \equiv f x$
2. **Right Identity:** $m \gg= \text{return} \equiv m$
3. **Associativity:** $(m \gg= f) \gg= g \equiv m \gg= (\lambda x \rightarrow f x \gg= g)$

1. **Associativity:** $(a <> b) <> c \equiv a <> (b <> c)$

$$\begin{aligned}
 (a <> b) <> c &= (a \gg= \lambda _ \rightarrow b) \gg= \lambda _ \rightarrow c \\
 &= a \gg= (\lambda x \rightarrow (\lambda _ \rightarrow b) x \gg= \lambda _ \rightarrow c) \quad (\text{Monad Associativity}) \\
 &= a \gg= (\lambda _ \rightarrow b \gg= \lambda _ \rightarrow c) \\
 &= a \gg= (\lambda _ \rightarrow (b <> c)) \\
 &= a <> (b <> c)
 \end{aligned}$$

2. Left Identity: $\text{mempty} <> a \equiv a$

$$\begin{aligned}
 \text{mempty} <> a &= \text{return } () \gg= \lambda_ \rightarrow a \\
 &= (\lambda_ \rightarrow a) () \quad (\text{Monad Left Identity}) \\
 &= a
 \end{aligned}$$

3. Right Identity: $a <> \text{mempty} \equiv a$

$$a <> \text{mempty} = a \gg= \lambda_ \rightarrow \text{return } ()$$

Since $a :: m ()$, the bound value inside the monadic computation is guaranteed to be $()$. Therefore, $\lambda_ \rightarrow \text{return } ()$ behaves identically to return .

$$\begin{aligned}
 a \gg= \lambda_ \rightarrow \text{return } () &\equiv a \gg= \text{return} \\
 &= a \quad (\text{Monad Right Identity})
 \end{aligned}$$